

## Database Integration Pattern

A database tool server exposes a few narrow, safe tools rather than open-ended SQL execution:

| Tool                  | Purpose                                                          |
|-----------------------|------------------------------------------------------------------|
| list_tables           | Discover available tables                                        |
| describe_table(table) | Inspect a table's columns and types                              |
| query(sql, params)    | Run a read-only SELECT/WITH query, parameterized and row-limited |

The AI is expected to:

1. Explore the schema first.
2. Then perform safe read-only queries.
3. Never perform writes.

## The Security Model

Six guards are enforced and tested — each stops a specific attack:

1. Read-Only Database Connection
  - SQLite opened in mode=ro.
  - Any write operation fails automatically.
2. Allow-Listed Operations
  - Only SELECT and WITH statements are allowed.
3. Single-Statement Enforcement
  - Queries containing ; are rejected.
  - Prevents stacked-query attacks.
4. Parameterized Queries
  - Uses ? placeholders and bound parameters.
  - Prevents SQL injection.
5. Result Limits
  - Limits returned rows (MAX\_ROWS).
  - Prevents excessive memory usage.
6. Identifier Validation
  - Table names are validated against actual database tables before being used.
  - Prevents injection through table or column names.

## Hands-On Lab

### Step 1 — seed\_db.py (Create the Sample Database)

Create seed\_db.py with the following code, then run it once to create store.db:

```
#!/usr/bin/env python3
```

```
"""Create a small SQLite database (store.db) for the Database Tool Server lab.
```

Run once before db\_server.py: python seed\_db.py

```
"""
```

```
import os
```

```
import sqlite3
```

```
from pathlib import Path
```

```
DB_PATH = Path(os.environ.get("DB_PATH", "./store.db")).resolve()
```

```
conn = sqlite3.connect(DB_PATH)
```

```
conn.executescript(  
    """
```

```
    """
```

```
    DROP TABLE IF EXISTS orders;
```

```
    DROP TABLE IF EXISTS products;
```

```
    CREATE TABLE products (  
        id    INTEGER PRIMARY KEY,
```

```
        name  TEXT NOT NULL,
```

```
        category TEXT,
```

```
        price REAL
```

```
    );
```

```
    CREATE TABLE orders (  
        id      INTEGER PRIMARY KEY,
```

```
        product_id INTEGER REFERENCES products(id),  
        qty        INTEGER,
```

```
        customer  TEXT
```

```
    );
```

```
    INSERT INTO products (name, category, price) VALUES
```

```
        ('Widget', 'hardware', 9.99),
```

```
        ('Gadget', 'hardware', 19.99),
```

```
        ('Manual', 'docs', 0.0);

INSERT INTO orders (product_id, qty, customer) VALUES
    (1, 3, 'alice'),
    (2, 1, 'bob'),
    (1, 5, 'carol');
"""
)
conn.commit()
conn.close()
print(f"Seeded database at {DB_PATH}")
```

Run the seed script:

```
python seed_db.py
```

## Step 2 — db\_server.py (The MCP Tool Server)

Create **db\_server.py** with the complete server code below:

```
"""Database Tool Server - narrow, read-only tools over a SQLite database."""
import os
import sqlite3
import sys
from pathlib import Path
from typing import Any

from pydantic import BaseModel

from mcp.server import MCPServer
from mcp.server.mcpserver.exceptions import ToolError
from mcp.types import ToolAnnotations

DB_PATH = Path(os.environ.get("DB_PATH", "./store.db")).resolve()
MAX_ROWS = int(os.environ.get("MAX_ROWS", "100"))
ALLOWED_PREFIXES = ("SELECT", "WITH")
READ_ONLY = ToolAnnotations(read_only_hint=True, destructive_hint=False,
open_world_hint=False)
```

```
mcp = MCPServer("Database Tool Server")

class Column(BaseModel):
    name: str
    type: str
    notnull: bool
    pk: bool

class QueryResult(BaseModel):
    columns: list[str]
    row_count: int
    max_rows: int
    rows: list[dict[str, Any]]

def _connect() → sqlite3.Connection:
    """Guard 1: open a READ-ONLY connection. Any write attempt fails at the driver level."""
    if not DB_PATH.exists():
        raise ToolError(f"Database not found at {DB_PATH}. Run: python seed_db.py")
    conn = sqlite3.connect(f"file:{DB_PATH.as_posix()}?mode=ro", uri=True)
    conn.row_factory = sqlite3.Row
    return conn

def _ensure_safe(sql: str) → str:
    """Guards 2 and 3: allow only a single read-only SELECT/WITH statement."""
    stripped = sql.strip().rstrip(";").strip()
    if ";" in stripped:
        raise ToolError("Only a single statement is allowed (no ';').")
    if not stripped.upper().startswith(ALLOWED_PREFIXES):
        raise ToolError("Only read-only SELECT/WITH queries are allowed.")
    return stripped
```

```
@mcp.tool(annotations=READ_ONLY)
def list_tables() → list[str]:
    """List the user tables in the database. Call this first to explore the schema."""
    conn = _connect()
    try:
        rows = conn.execute(
            "SELECT name FROM sqlite_master "
            "WHERE type='table' AND name NOT LIKE 'sqlite_%' ORDER BY name"
        ).fetchall()
    finally:
        conn.close()
    return [r["name"] for r in rows]

@mcp.tool(annotations=READ_ONLY)
def describe_table(table: str) → list[Column]:
    """Return the column definitions (name, type, not-null, primary-key) for a table."""
    conn = _connect()
    try:
        # Guard 6: the identifier must be a real table (PRAGMA cannot be parameterized).
        valid = {r["name"] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'")}
        if table not in valid:
            raise ToolError(f"Unknown table: '{table}'. Call list_tables to see valid names.")
        cols = conn.execute(f"PRAGMA table_info('{table}')").fetchall()
    finally:
        conn.close()
    return [Column(name=c["name"], type=c["type"], notnull=bool(c["notnull"]), pk=bool(c["pk"]))
for c in cols]

@mcp.tool(annotations=READ_ONLY)
def query(sql: str, params: list[str] | None = None) → QueryResult:
```

```
"""Run a read-only SELECT/WITH query.
```

Use ? placeholders in `sql` and pass values in `params` to stay injection-safe,  
e.g. sql="SELECT \* FROM products WHERE category = ?", params=["hardware"].

At most MAX\_ROWS rows are returned.

```
"""
```

```
safe_sql = _ensure_safe(sql)
```

```
conn = _connect()
```

```
try:
```

```
    cur = conn.execute(safe_sql, params or [])      # Guard 4: bound parameters
```

```
    cols = [d[0] for d in cur.description] if cur.description else []
```

```
    rows = [dict(r) for r in cur.fetchmany(MAX_ROWS)] # Guard 5: row limit
```

```
except sqlite3.Error as exc:
```

```
    raise ToolError(f"Query failed: {exc}")
```

```
finally:
```

```
    conn.close()
```

```
return QueryResult(columns=cols, row_count=len(rows), max_rows=MAX_ROWS, rows=rows)
```

```
if __name__ == "__main__":
```

```
    print(f"Database Tool Server (read-only) using {DB_PATH}", file=sys.stderr)
```

```
    mcp.run()
```

### Step 3 — client.py (Test All Six Guards)

Create client.py. It starts db\_server.py over stdio and runs eight scenarios — four that should succeed and four attacks that must be blocked:

```
import asyncio
```

```
import sys
```

```
from mcp import Client, StdioServerParameters
```

```
server = StdioServerParameters(command=sys.executable, args=["db_server.py"])
```

```
def show(result) → None:
```

```
"""Print rows for a successful query, or the error message the model would see."""
if result.is_error:
    print("Blocked:", result.content[0].text)
elif result.structured_content and "rows" in result.structured_content:
    for row in result.structured_content["rows"]:
        print(row)
else:
    print(result.structured_content)

async def main():
    async with Client(server) as client:
        print("\n=== Available Tools ===")
        for tool in (await client.list_tools()).tools:
            print(tool.name)

        print("\n=== 1. List Tables ===")
        show(await client.call_tool("list_tables", {}))

        print("\n=== 2. Describe a Table ===")
        show(await client.call_tool("describe_table", {"table": "products"}))

        print("\n=== 3. SELECT Query ===")
        show(await client.call_tool("query", {"sql": "SELECT name, price FROM products ORDER BY
price DESC"}))

        print("\n=== 4. Parameterized Query ===")
        show(await client.call_tool("query", {
            "sql": "SELECT * FROM products WHERE category = ?",
            "params": ["hardware"],
        }))

        print("\n=== 5. Write Attempt (should be blocked) ===")
        show(await client.call_tool("query", {"sql": "INSERT INTO products(name) VALUES ('x')"}))
```

```

print("\n=== 6. Stacked Query Injection (should be blocked) ===")
show(await client.call_tool("query", {"sql": "SELECT 1; DROP TABLE products"}))

print("\n=== 7. Table Name Injection (should be blocked) ===")
show(await client.call_tool("describe_table", {"table": "products; DROP TABLE products"}))

print("\n=== 8. Write hidden in a WITH (blocked by the read-only connection) ===")
show(await client.call_tool("query", {
    "sql": "WITH x AS (SELECT 1) DELETE FROM products WHERE id IN (SELECT * FROM x)",
}))

if __name__ == "__main__":
    asyncio.run(main())

```

Run it (from the folder that contains store.db):

```
python client.py
```

Expected output (abridged):

```

=== 1. List Tables ===
{'result': ['orders', 'products']}

=== 3. SELECT Query ===
{'name': 'Gadget', 'price': 19.99}
{'name': 'Widget', 'price': 9.99}
{'name': 'Manual', 'price': 0.0}

=== 5. Write Attempt (should be blocked) ===
Blocked: Error executing tool query: Only read-only SELECT/WITH queries are allowed.

=== 6. Stacked Query Injection (should be blocked) ===
Blocked: Error executing tool query: Only a single statement is allowed (no ';').

=== 7. Table Name Injection (should be blocked) ===
Blocked: Error executing tool describe_table: Unknown table: 'products; DROP TABLE products'.
Call list_tables to see valid names.

```

```
=== 8. Write hidden in a WITH (blocked by the read-only connection) ===
```

```
Blocked: Error executing tool query: Query failed: attempt to write a readonly database
```

Scenario 8 shows why guard 1 matters: the allow-list accepts the statement because it starts with WITH, but the read-only connection still refuses the write. Each guard is a separate layer.

### Done When...

- seed\_db.py created store.db; list\_tables returns {"result": ["orders", "products"]} as structured content
- A SELECT query returns rows; describe\_table products returns typed column objects (name, type, notnull, pk)
- INSERT / UPDATE / DELETE / DROP all return isError: true (allow-list)
- SELECT 1; DROP TABLE products returns isError: true (single-statement guard)
- An injection-style table name returns isError: true (identifier validation)
- A write hidden in a WITH statement fails with "attempt to write a readonly database" (read-only connection)
- Note all six security guards and which attack each one stops

### We can Verify in the Inspector also:

The Inspector CLI needs Node.js 22.19 or later. Run each command below and confirm the expected output:

#### List tables

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
  --method tools/call --tool-name list_tables
```

Expected output:

```
"structuredContent": {"result": ["orders", "products"]}
```

#### Run a SELECT query

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \
  --method tools/call --tool-name query \
  --tool-arg "sql=SELECT name, price FROM products ORDER BY price DESC"
```

Expected output:

```
{"columns": ["name", "price"], "row_count": 3, "max_rows": 100,
"rows": [{"name": "Gadget", "price": 19.99}, {"name": "Widget", "price": 9.99},
{"name": "Manual", "price": 0.0}]}
```

**Write blocked by allow-list**

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \  
  --method tools/call --tool-name query \  
  --tool-arg "sql=INSERT INTO products (name) VALUES ('x')"
```

Expected output:

```
"text": "Error executing tool query: Only read-only SELECT/WITH queries are allowed.",  
"isError": true
```

**Stacked-query injection blocked**

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \  
  --method tools/call --tool-name query \  
  --tool-arg "sql=SELECT 1; DROP TABLE products"
```

Expected output:

```
"text": "Error executing tool query: Only a single statement is allowed (no ';' ).",  
"isError": true
```

**Injection-style table name blocked**

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \  
  --method tools/call --tool-name describe_table \  
  --tool-arg "table=products; DROP TABLE products"
```

Expected output:

```
"text": "Error executing tool describe_table: Unknown table: 'products; DROP TABLE products'.  
Call list_tables to see valid names.",  
"isError": true
```

**Parameterized query (injection-safe filtering)**

Use ? placeholders with a params list. The value is bound by the driver, never concatenated into SQL:

```
npx @modelcontextprotocol/inspector --cli mcp run db_server.py \  
  --method tools/call --tool-name query \  
  --tool-arg "sql=SELECT * FROM products WHERE category = ?" \  
  --tool-arg "params=[\"hardware\"]"
```

Expected output:

```
{"columns": ["id", "name", "category", "price"], "row_count": 2, "max_rows": 100,  
"rows": [{"id": 1, "name": "Widget", "category": "hardware", "price": 9.99},  
{"id": 2, "name": "Gadget", "category": "hardware", "price": 19.99}]}
```